UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

CARRERA DE INGENIERÍA DE SOFTWARE

CATÁLOGO DE LA BIBLIOTECA FISEI

Árbol general n-ario y árbol binario de búsqueda implementados en Java

Campo	Información
Asignatura	Estructura de Datos
Nivel	Tercero – Ingeniería de Software
Modalidad	Individual
Estudiante	Fernando Villacres
Docente	Completar antes de entregar
Fecha	22 de octubre de 2026

Informe técnico y borrador de evidencias del proyecto

1. Resumen del proyecto

Se desarrolló una aplicación de consola en Java para organizar el catálogo de la Biblioteca FISEI. La jerarquía Biblioteca → Área → Estante se representa con un árbol general. Cada estante mantiene sus libros en un árbol binario de búsqueda (ABB) ordenado por un código entero positivo, sin duplicados. Las operaciones principales se implementan recursivamente.

El proyecto incorpora validaciones del dominio, recorridos preorden/inorden/posorden con traza, búsqueda, inserción, modificación, eliminación por casos, consultas por rango, ruta de búsqueda y verificación de la propiedad ABB. Incluye menú interactivo, datos iniciales y una suite de pruebas.

2. Organización e implementación

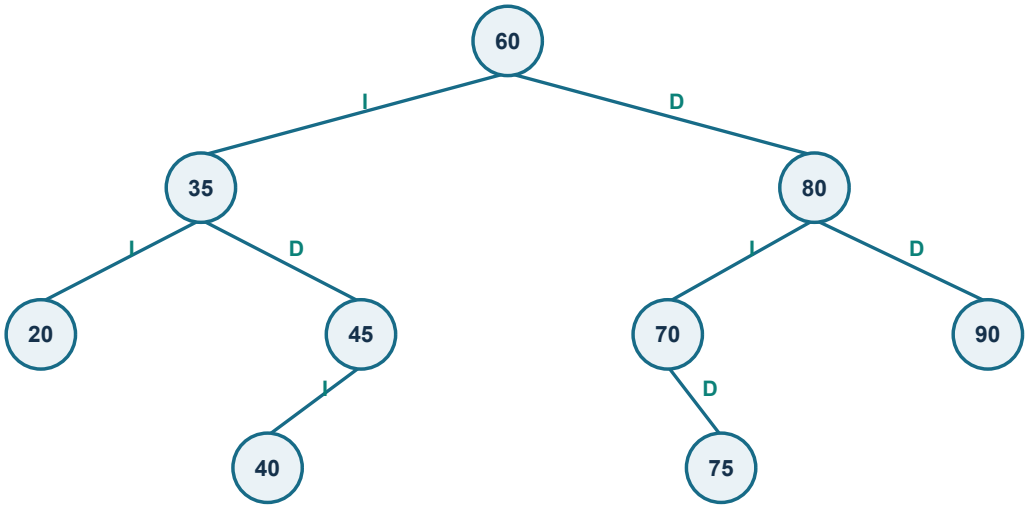
Paquete / clase	Responsabilidad
enums.TipoUnidad, TipoRecorrido	Tipos permitidos de unidad y estrategias de recorrido.
modelo.Libro	Código inmutable; título, autor y año validados (1900–2026).
modelo.UnidadBiblioteca	Código normalizado en mayúsculas, nombre y tipo.
negocio.NodoUnidad, ArbolBiblioteca	Jerarquía n-aria y consultas recursivas sobre unidades.
negocio.NodoLibro, CatalogoABB	Nodos y operaciones recursivas del ABB de cada estante.
app.Main, DatosPrueba	Menú de consola y construcción de la estructura inicial.
app.PruebasBiblioteca	Ejecución automatizada de 12 grupos de validación.

La inserción en la jerarquía solo admite BIBLIOTECA → AREA → ESTANTE y no permite códigos repetidos. En el ABB, la eliminación de un nodo con dos hijos utiliza el sucesor inorden (mínimo del subárbol derecho), preservando el orden del árbol. La reconstrucción opcional desde preorden consume la secuencia directamente con límites, sin insertar los valores uno por uno.

3. Estructura inicial de la biblioteca

```
BIB Biblioteca FISEI
  SW Software
    SW-E1 Estante 1 de Software: 60, 35, 80, 20, 45, 70, 90, 40, 75
    SW-E2 Estante 2 de Software: 15, 10, 25
  TI Tecnologías de la Información
    TI-E1 Estante 1 de TI: 50
  TEL Telecomunicaciones
```

La estructura contiene 7 unidades, 4 hojas, altura 2 aristas y 13 libros. El siguiente diagrama representa el ABB inicial del estante SW-E1; cada arista se marca con I (izquierdo) o D (derecho).


60
I
D
35
80
I
D
I
D
20
45
70
90
I
D
40
75

4. Recorridos y consultas del ABB

Operación	Resultado esperado / comprobado
Preorden	60 35 20 45 40 80 70 75 90
Inorden	20 35 40 45 60 70 75 80 90
Posorden	20 40 45 35 75 70 90 80 60
caminoBusqueda(42)	[60, 35, 45, 40] (42 no está; ruta hasta el punto de búsqueda)
contarEnRango(40, 75)	5: códigos 40, 45, 60, 70, 75
Código duplicado	insertar(60) devuelve false

La traza informa la entrada a cada nodo, la visita según el orden elegido y cada hijo vacío como “null → regresa”. La secuencia del recorrido aparece como última línea.

Ejemplo de traza de preorden

```
entra 60
VISITA 60
  entra 35
  VISITA 35
    entra 20
    VISITA 20
      null -> regresa
      null -> regresa
    entra 45
    VISITA 45
      entra 40
      VISITA 40
        null -> regresa
        null -> regresa
      null -> regresa
  entra 80
  VISITA 80
    entra 70
    VISITA 70
      null -> regresa
      entra 75
      VISITA 75
        null -> regresa
        null -> regresa
    entra 90
    VISITA 90
      null -> regresa
      null -> regresa
Secuencia PREORDEN: 60 35 20 45 40 80 70 75 90
```

5. Eliminación y validación estructural

Para comprobar los tres casos de eliminación se inserta 42 en el ABB inicial y después se eliminan 20 (hoja), 70 (un hijo, 75) y 60 (dos hijos). El sucesor inorden de 60 es 75. El preorden final comprobado es: 75 35 45 40 42 80 90.

La validación ABB examina cada nodo con límites mínimo y máximo heredados de todos sus antecesores; así también detecta un nieto que viole el límite de un abuelo, no solo errores entre padre e hijo.

6. Pruebas ejecutadas

Compilado con OpenJDK 17. La ejecución actual de PruebasBiblioteca finalizó correctamente: 12/12 pruebas OK. Se cubrieron biblioteca vacía e inicial, niveles, reglas de inserción, recorridos, duplicados, camino y rango, los tres casos de eliminación, modificación, operaciones con códigos inexistentes, validaciones, ABB inválido y reconstrucción desde preorden.

Salida comprobada:

```
OK  biblioteca vacía
OK  estructura inicial
OK  niveles y búsqueda sin distinguir mayúsculas
OK  reglas de inserción y códigos únicos
OK  recorridos del estante SW-E1
OK  duplicado, camino y rango
OK  eliminación de hoja, un hijo y dos hijos
OK  modificación conserva código y valida cambios
OK  operaciones sobre libro inexistente
OK  validaciones de datos de libro
OK  ABB inválido por nieto mal ubicado
OK  reconstrucción directa desde preorden
12/12 pruebas OK
```

7. Análisis de complejidad

Operación	Complejidad	Observación
Búsqueda en ABB	O(h)	h es la altura; en un árbol balanceado es O(log n), pero en el peor caso llega a O(n).
Búsqueda en árbol general	O(N)	Puede ser necesario revisar las N unidades porque no hay orden de búsqueda.
Recorrido ABB	O(n)	Cada nodo se visita una vez.
Altura ABB degenerado	h = n - 1	Insertar códigos ya ordenados forma una cadena y empeora búsquedas e inserciones.

8. Reflexión técnica — borrador para revisar y personalizar

¿Por qué buscar en el ABB es O(h) y en el árbol general O(N)? ¿Qué ocurre si se insertan códigos ordenados?

En un ABB, cada comparación permite continuar solo por el hijo izquierdo o derecho, por lo que se recorre como máximo un camino de la raíz a un nodo: su costo es proporcional a la altura h. En un árbol general sin orden de búsqueda, un código puede estar en cualquier rama y, en el peor caso, hay que revisar todas las N unidades. Si los códigos se insertan ordenados en un ABB sin balanceo, cada nuevo valor cae en el mismo lado y el árbol se degenera en una cadena; h puede crecer hasta n−1 y las operaciones pasan a O(n).

¿Por qué eliminar un nodo con dos hijos usando el sucesor inorden?

El sucesor inorden es el menor código mayor que el nodo eliminado: el mínimo de su subárbol derecho. Al reemplazar el dato por ese sucesor y retirar su posición original, los valores del subárbol izquierdo siguen siendo menores y los restantes del derecho mayores. Elegir cualquier nodo del subárbol derecho no garantiza que sea el menor ni que el orden BST se conserve.

¿Cómo podría usarse un árbol general o ABB en SmartETL?

Un árbol general puede representar una canalización ETL jerárquica (proyecto, fuentes, etapas y tareas), facilitando recorrer sus dependencias o mostrar su estructura. Un ABB podría indexar registros por una clave entera para búsquedas y consultas por intervalos. En producción, si se requiere rendimiento predecible ante entradas ordenadas, convendría usar un árbol balanceado o un índice de base de datos; la elección depende de las necesidades de actualización, rango y volumen.

Nota: estas respuestas son un borrador técnico. Revísalas y expresa tu comprensión con tus propias palabras antes de presentar el PDF como trabajo individual.

9. Repositorio y control de versiones

Repositorio público: github.com/fercho123909/Deber-Arboles-VillacresFernando

Rama	Estado / commits
main	Documentación inicial (f0a7868).
desarrollo	Implementación y CI; commits 298bbbd, 3befbc3, e8bd647 y e9b3ee6.
Pull request a main	Pendiente de crear e integrar; no se afirma que exista.

El workflow de GitHub Actions está configurado para compilar y ejecutar las pruebas en pushes a main/desarrollo y en pull requests hacia main. La verificación local de este informe fue 12/12 pruebas OK.

10. Lista de pendientes para la entrega final

Pendiente	Acción personal necesaria
Códigos y traza personal	Derivar localmente los códigos a partir de la regla del enunciado; completar el ABB extendido y sus recorridos en TRAZAS.md. No publicar la cédula.
Pull request	Crear desde desarrollo hacia main y completar la integración requerida.
Capturas reales	Tomar capturas propias de ejecución, pruebas, historial de commits y pull request. Este informe no presenta imágenes simuladas como capturas.
Reflexión	Revisar y adaptar el borrador de la sección 8 a la experiencia propia.
PDF final	Actualizar este documento con las evidencias reales y verificar el enlace al repositorio antes de subirlo a Moodle.

Instrucciones para recompilar y ejecutar: consultar el README del repositorio. La aplicación y las pruebas se compilaron con Java 17.